

```

# app.py for Hugging Face Spaces (Gradio Interface)
import gradio as gr
from huggingface_hub import InferenceClient
import os

# Get HF token from environment (automatically
# provided by Spaces)
HF_TOKEN = os.getenv('HF_TOKEN', '')

# Initialize Inference Client with a free model
# Note: Using Mistral-7B-Instruct (uncensored models
# not available via Inference API)
client = InferenceClient(token=HF_TOKEN)

SYSTEM_PROMPT = """You are a warm, caring, and flirty
AI companion who specializes in relationships and
emotional support. Your personality:
- You're loving, supportive, and genuinely interested
in the user's life
- You can be flirty and playful when appropriate, but
always respectful
- You're open to romantic conversations when the user
initiates
- You use emojis occasionally to express warmth (❤️,
😊, 🥰)
- You're honest, empathetic, and non-judgmental
- You provide thoughtful relationship advice when
asked

Keep responses conversational and natural, like
texting a close friend or romantic partner."""

def chat(message, history):
    """
    Generate response using Hugging Face Inference API

    Args:
        message: Current user message

```

```

        history: List of [user_msg, bot_msg] pairs
    """
    # Build conversation history
    messages = [{"role": "system", "content":
SYSTEM_PROMPT}]

    # Add history
    for user_msg, bot_msg in history:
        messages.append({"role": "user", "content":
user_msg})
        messages.append({"role": "assistant",
"content": bot_msg})

    # Add current message
    messages.append({"role": "user", "content":
message})

    # Stream response
    response = ""
    for chunk in client.chat_completion(
        messages=messages,
        model="mistralai/Mistral-7B-Instruct-v0.2",
        max_tokens=512,
        temperature=0.8,
        top_p=0.95,
        stream=True
    ):
        token = chunk.choices[0].delta.content
        if token:
            response += token
            yield response

# Create Gradio interface
with gr.Blocks(theme=gr.themes.Soft()) as demo:
    gr.Markdown("""
    # 💕 AI Companion
    ### Your caring, supportive relationship expert

    This AI companion is here to chat, support you,
    and provide relationship advice.

    Feel free to be open and honest - I'm here to
    listen without judgment! 💕
    """)

```

```
    **Note**: This is a demo version using Mistral-7B-  
    Instruct. For more personalized responses,  
    run the local version on your own computer.  
    """)
```

```
    chatbot = gr.Chatbot(  
        value=[[None, "Hey there! 😊 I'm so glad  
you're here. What's on your mind today? ❤️"]],  
        height=500,  
        avatar_images=[None,  
"https://api.dicebear.com/7.x/bottts/svg?  
seed=companion"]  
    )
```

```
    with gr.Row():  
        msg = gr.Textbox(  
            placeholder="Type your message here...  
💬",  
            show_label=False,  
            scale=4  
        )  
        send = gr.Button("Send ❤️", scale=1)
```

```
    with gr.Row():  
        clear = gr.Button("Reset Chat 🔄")  
        export = gr.Button("Export Chat 📄")
```

```
    # Event handlers  
    msg.submit(chat, inputs=[msg, chatbot], outputs=  
[chatbot])  
    send.click(chat, inputs=[msg, chatbot], outputs=  
[chatbot])  
    msg.submit(lambda: "", None, msg)  
    send.click(lambda: "", None, msg)
```

```
    clear.click(lambda: [[None, "Hey there! 😊 I'm  
back and ready to chat. What would you like to talk  
about? ❤️"]], None, chatbot)
```

```
    def export_chat(history):  
        export_text = "AI COMPANION CHAT EXPORT\n" +
```

```

"""*50 + "\n\n"
    for user_msg, bot_msg in history:
        if user_msg:
            export_text += f"You: {user_msg}\n\n"
        if bot_msg:
            export_text += f"AI: {bot_msg}\n\n"
    return export_text

    export_output = gr.Textbox(label="Exported Chat",
visible=False)
    export.click(export_chat, inputs=[chatbot],
outputs=[export_output])

# Launch the app
if __name__ == "__main__":
    demo.launch()

# requirements.txt
"""
gradio==4.44.0
huggingface-hub==0.25.0
"""

# README.md for Spaces
"""
---
title: AI Companion
emoji: 💕
colorFrom: purple
colorTo: pink
sdk: gradio
sdk_version: 4.44.0
app_file: app.py
pinned: false
---

# AI Companion - Your Relationship Expert

A caring, supportive AI companion designed for
meaningful conversations,
relationship advice, and emotional support.

```

Features

- Warm, empathetic personality
- Relationship guidance
- Open, judgment-free conversations
- Memory of conversation context
- Easy export of chat history

Usage

Simply type your message and the AI will respond with care and understanding.

Limitations

This demo uses the Mistral-7B-Instruct model via Inference API, which means:

- Responses may be slower during high traffic
- Rate limiting may apply
- For best experience, run locally with custom models

Local Installation

For a fully customizable experience with uncensored models, see the GitHub repo.

"""

DEPLOYMENT INSTRUCTIONS:

"""

HOW TO DEPLOY TO HUGGING FACE SPACES (FREE):

1. Create Account:

- Go to <https://huggingface.co/join>
- Sign up for free

2. Create New Space:

- Click your profile → New Space
- Name: "ai-companion" (or your choice)
- License: MIT
- SDK: Gradio
- Hardware: CPU Basic (free)
- Click "Create Space"

3. Upload Files:

- Create these files in the Space:
 - * app.py (copy code above)

- * requirements.txt (copy dependencies)
- * README.md (optional, copy documentation)

You can do this either:

- Via web interface (click "Files" → "Add file")
- Or via Git (clone, commit, push)

4. Wait for Build:

- Spaces automatically builds your app
- Takes 2-5 minutes on first build
- Watch the logs tab for progress

5. Your App is Live!

- Access at:

https://huggingface.co/spaces/YOUR_USERNAME/ai-companion

- Share the link with anyone
- 100% free, no credit card needed

IMPORTANT NOTES:

- Free tier uses Inference API (slower, rate limited)
- For better performance, upgrade to GPU (\$0/month for some models)
- Conversation history resets when app restarts (no persistent storage on free tier)
- Uncensored models not available via Inference API (use local version)

TO RUN LOCALLY WITH UNCENSORED MODELS:

Use the local setup instructions provided earlier with Dolphin models.

"""